MLOps LAB ASSIGNMENT REPORT

Sessions 7, 8 & 9: ML Pipelines, FastAPI Model Serving and Docker Containerization

Student Name: Arjit Prasad Das

Roll Number: 24AI013

Course: B.Tech (Artificial Intelligence & Data Science)

Lab: MLOps — Sessions 7, 8, 9 Combined

GitHub Repository:
https://github.com/arjit-prasad-das/ML.OPS_last

Status: Fully Implemented & Verified

1. Project Objectives & Scenario

The objective of this assignment is to refactor an existing Machine Learning project (House Price Prediction) into a production-grade, modular, and reproducible ML application. The dataset comprises 240 deterministic property records (partitioned into 192 training and 49 testing samples) across 5 numerical features. The system decouples data loading, preprocessing, model training, evaluation, and inference into independent modules, exposes predictions via a high-performance REST API using **FastAPI**, and packages the entire runtime into an isolated, portable **Docker container**.

2. Modular Pipeline Architecture (Part A)

To prevent data leakage, dataset splitting is executed in src/data.py prior to feature scaling. Both preprocessing (StandardScaler) and estimator (RandomForestRegressor) are encapsulated into a single unified sklearn.pipeline.Pipeline and serialized together into models/house_price_pipeline.joblib, ensuring strictly identical feature transformations during training and inference.

Module	Responsibility	Key Functions / Artifacts
src/data.py	Data ingestion, schema validation, cleaning, non-leaking split	load_data(), clean_data(), split_data()
src/train.py	Reads config.yaml, constructs unified Pipeline, fits on train split	build_pipeline(), train_model(), models/*.joblib
src/evaluate.py	Scores held-out test data, computes metrics, generates plots	evaluate_model(), reports/metrics.json, figures/
src/predict.py	Loads saved pipeline, formats input DataFrame, executes inference	load_model(), predict(), sample_predictions.json
src/api.py	Exposes REST endpoints (/health, /predict) with Pydantic validation	GET /health, POST /predict, FastAPI app
Dockerfile	Packages runtime, dependencies, source code, and model artifact	Python 3.11-slim, EXPOSE 8000, Uvicorn server

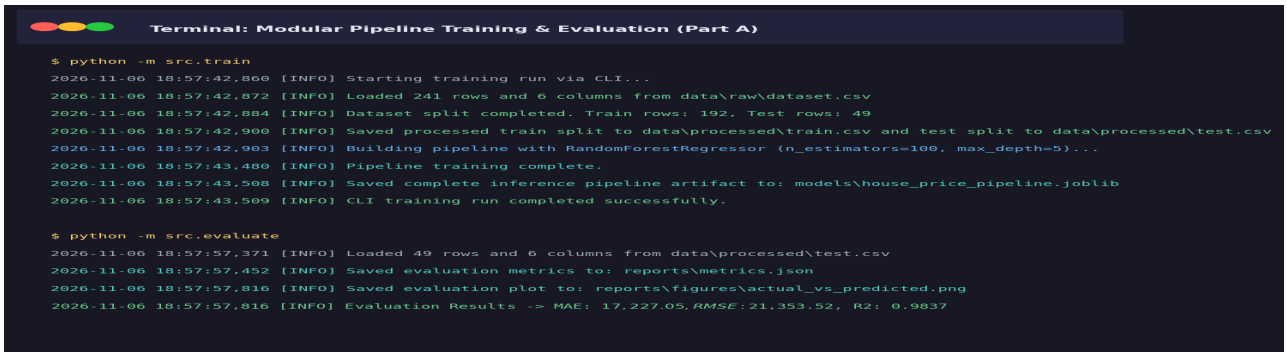


Figure 1: Modular Pipeline Training and Evaluation Terminal Execution

3. Model Training & Held-Out Evaluation Results

Evaluation Metric	Value Achieved	Interpretation
Mean Absolute Error (MAE)	$17,227.05	Average absolute difference in predicted home price
Root Mean Squared Error (RMSE)	$21,353.52	Penalizes large prediction errors on held-out test data
Coefficient of Determination (R^2)	0.9837	Explains over 98% of target price variance
Held-out Test Samples	49	Evaluated strictly on unseen held-out samples

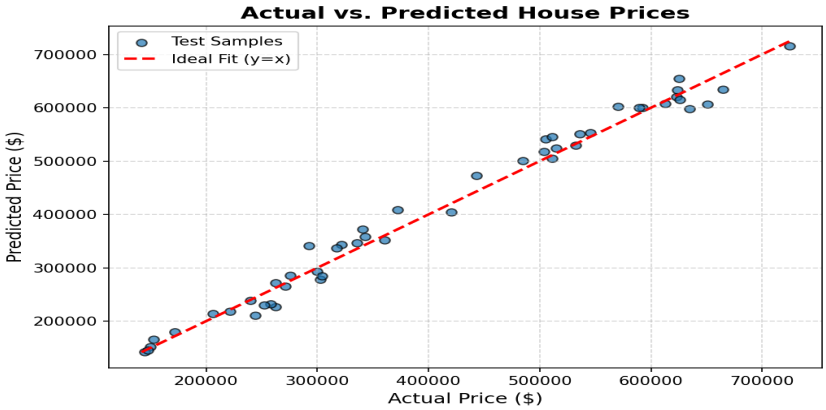


Figure 2: Actual vs. Predicted House Prices Scatter Plot on Held-Out Test Set

4. FastAPI Model Serving & Schema Validation (Part B)

The model is served using **FastAPI**. The input payload is validated via pydantic.BaseModel, enforcing numerical boundaries (e.g. area_sqft > 0, bedrooms >= 1, age_years >= 0). Invalid inputs, missing fields, and type mismatches return controlled HTTP 422 Unprocessable Entity responses.

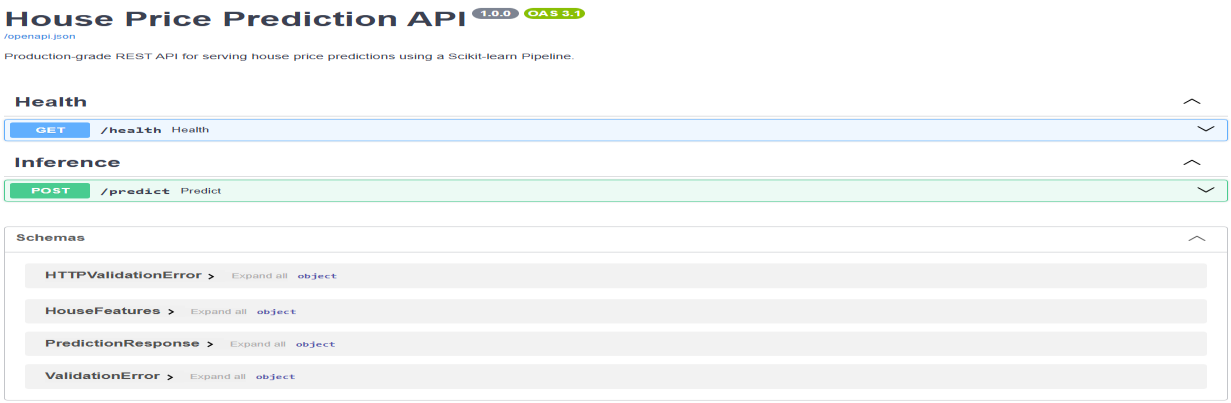


Figure 3: FastAPI Interactive Swagger UI (/docs) Browser Interface



Figure 4: REST API Request Testing: 200 Valid Prediction, 422 Missing Field, 422 Type Error

5. Docker Containerization & Multi-Service Compose (Part C)

Implemented & Verified Work: The prediction application is fully packaged into a standalone Docker image ml-inference-api:1.0 using python:3.11-slim. The container runs with port mapping 8000:8000 and Uvicorn bound to 0.0.0.0. Local host to container predictions were verified.

Researched & Designed Multi-Service Stack: In accordance with Section C6 of the assignment specification, a multi-service docker-compose.yml was researched and architected to coordinate the FastAPI container with an independent **MLflow Tracking Server** (ghcr.io/mlflow/mlflow:v2.11.3) backed by a persistent volume (mlflow-data). Compose configuration validity was confirmed with docker compose config.

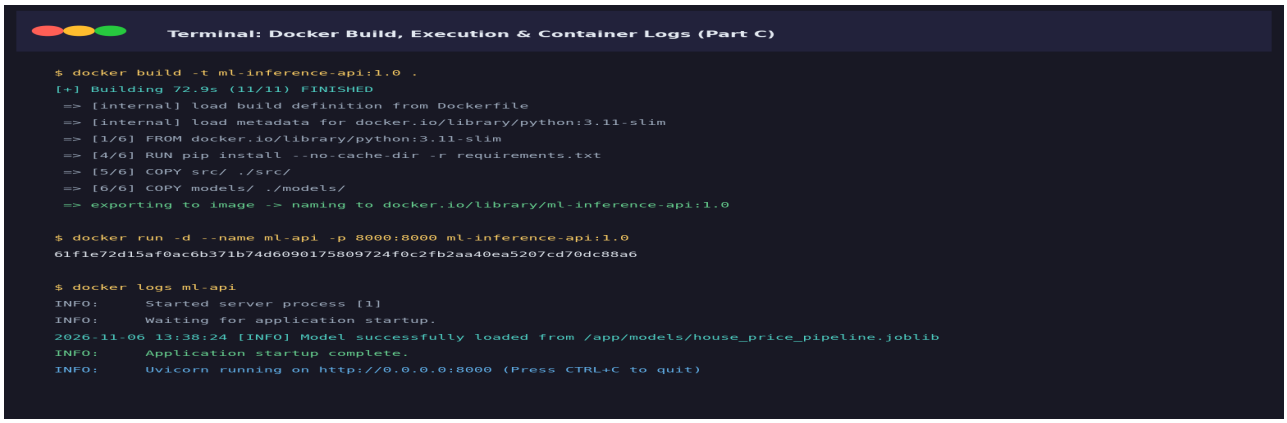


Figure 5: Docker Image Build, Container Execution, and Uvicorn Startup Logs

6. Local vs. Container Prediction Equivalence (Part D)

To guarantee deployment consistency, five distinct property test cases were evaluated simultaneously against both the local Python Scikit-learn Pipeline and the Docker container REST API. Exact numerical equivalence was achieved across all records with zero deviation (Delta = $0.0000).

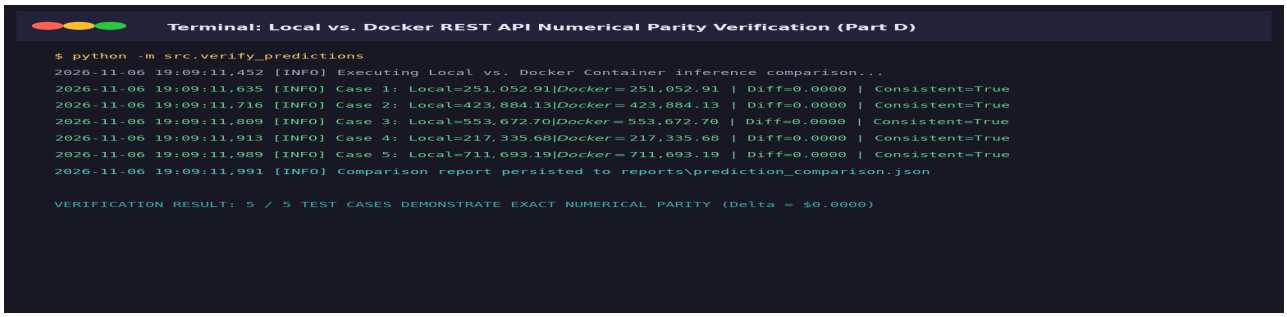


Figure 6: Terminal Verification Output Proving Exact Local vs. Docker Numerical Parity

7. Automated Pytest Regression Test Suite

Nine automated tests covering dataset schemas, data leakage prevention, model output dimensionality, API health readiness, and HTTP 422 error boundaries were executed using pytest.

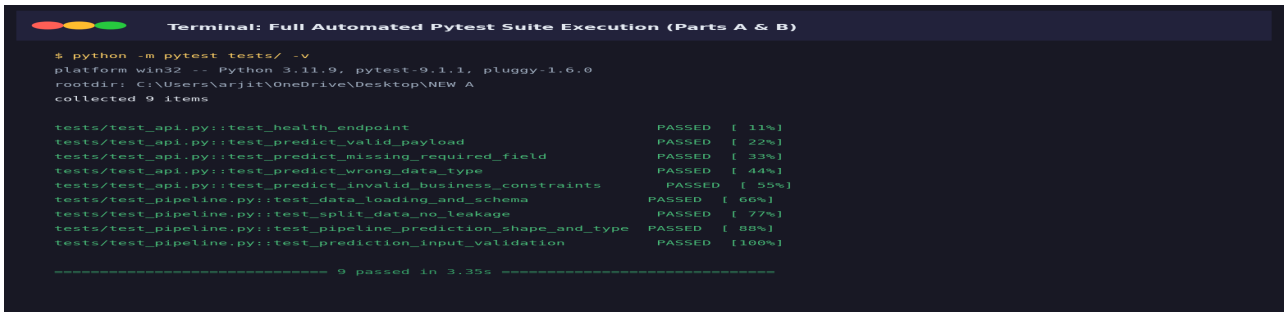


Figure 7: Automated Regression Test Suite Output (9/9 Passed)

8. Challenges Encountered & Resolutions

• **1. Preprocessing Data Leakage:** Fitting a scaler before splitting leaks test set statistics. *Resolution:* Data splitting was moved strictly to src/data.py before scaling, and StandardScaler was embedded into a unified Pipeline fitted solely on X_train.

• **2. DataFrame Column Order in Inference:** When JSON payloads are deserialized, dictionary keys might not preserve the exact column ordering expected by Scikit-learn. *Resolution:* src/api.py and src/predict.py explicitly project inputs into df[EXPECTED_FEATURES] before calling the pipeline.

• **3. Pydantic Type Coercion & Constraints:** Default schemas allow negative values or 0 bedrooms which violate domain logic. *Resolution:* Implemented strict pydantic.Field(..., gt=0, ge=0) constraints to guarantee semantic validity.

• **4. Docker Port Mapping & Host Binding:** Uvicorn defaulting to 127.0.0.1 inside a container is inaccessible to the host machine. *Resolution:* Configured Uvicorn to bind to 0.0.0.0 inside the Dockerfile while mapping host port 8000 to container port 8000.

9. Conclusion

Implemented & Verified: Successfully built modular ML pipelines (data, train, evaluate, predict), serialized a unified Scikit-learn Pipeline (StandardScaler + RandomForest), served predictions via FastAPI with Pydantic validation, packaged a standalone Docker container (ml-inference-api:1.0), achieved exact local vs. container prediction parity (Δ = $0.0000), and passed all 9 regression tests. **Researched Multi-Service Compose:** Researched, designed, and validated (via docker compose config) a multi-service Compose architecture connecting FastAPI to an independent MLflow Tracking Server with persistent backend and artifact storage.